Enterprise Sales / CRM Research Agent

System Architecture, MCP Protocol Specification & LangGraph State Machine Guide

Author: Forward Deployed Engineering (FDE) Team

Repository: github.com/devanshudhoble32/Crm_agent-

Core Frameworks: LangGraph, Anthropic MCP, FastMCP, Python 3.13

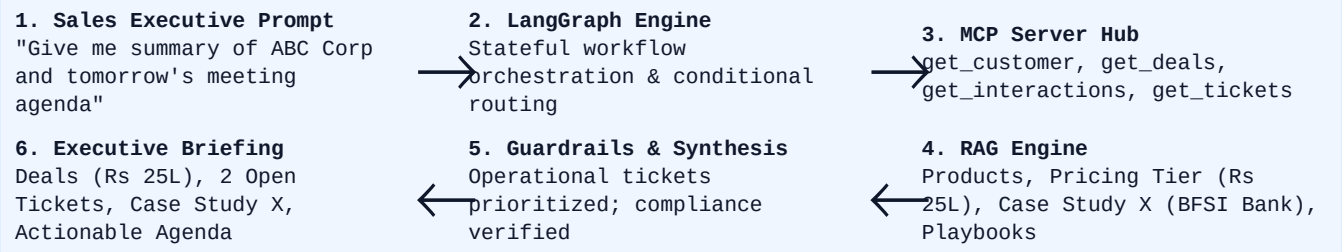
Target Domain: Enterprise BFSI, Tier-1 Account Strategy & Deal Acceleration

1. Executive Summary & Forward Deployed Engineering Vision

Modern enterprise sales professionals spend between 30 to 45 minutes manually preparing for client meetings. They scramble across multiple disconnected siloed systems: CRM records for account owners, billing pipelines for deal values, email/calendar archives for interaction history, and IT support ticketing platforms (e.g. Jira, Zendesk) for operational friction. Walking into an executive meeting unaware of ongoing staging/sandbox outages or unresolved high-priority support tickets severely damages executive credibility.

The **Sales / CRM Research Agent** solves this high-value enterprise problem. Given a simple conversational prompt—such as *"Give me a summary of customer ABC and tell me what I should discuss in tomorrow's meeting"*—the system autonomously coordinates live data retrieval across systems via the **Model Context Protocol (MCP)**, synthesizes targeted internal knowledge using **RAG** (products, pricing, peer case studies, playbooks), and executes a deterministic **LangGraph** state machine to produce an actionable, C-level executive briefing with prioritized agenda recommendations.

End-to-End Information Flow



2. Model Context Protocol (MCP): Server & Client Architecture

Model Context Protocol (MCP) is an open standard introduced by Anthropic that standardizes how LLM applications connect to external tools, data sources, and enterprise context. Instead of ad-hoc REST integrations and brittle function wrappers, MCP establishes a formal client-server architecture with structured JSON-RPC 2.0 messages, typed schemas, and capability negotiation.

2.1 Transportation Layer Analysis: stdio vs. SSE vs. Streamable HTTP

The MCP specification supports multiple transport mechanisms depending on the isolation and network requirements:

Transport	Mechanism	Best Use Case	Data Framing
`stdio` (Standard I/O)	Direct OS pipes (`stdin`/`stdout`). Subprocess spawned by host.	Local CLI, desktop IDEs, embedded sidecars, air-gapped runtimes.	`Line-delimited JSON-RPC 2.0 frames over stdout/stdin.`
`SSE` (Server-Sent Events)	HTTP POST for requests; persistent Server-Sent Events stream for responses.	Distributed microservices, Docker containers, multi-tenant Kubernetes clouds.	`W3C SSE stream (`event: message`, `data: {...}`) + HTTP POST.`
`Streamable HTTP`	Chunked HTTP 1.1/2 transfer encoding with bi-directional streaming.	High-throughput server-to-server gateways with proxy support.	`Transfer-Encoding: chunked with JSON payload streams.`

2.2 How the Local Server is Created & Registered

In our implementation (`mcp_server/server.py`), the server is built using the high-performance **FastMCP** library. Each tool is registered with typed signatures, docstrings, and JSON Schema definitions automatically exposed to clients:

```
# mcp_server/server.py
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("Sales-CRM-MCP-Server")

@mcp.tool()
def get_customer(customer_name_or_id: str) -> dict: ...

@mcp.tool()
def get_deals(customer_id: str) -> dict: ...

@mcp.tool()
def get_previous_interactions(customer_id: str, limit: int = 5) -> dict: ...

@mcp.tool()
def get_support_tickets(customer_id: str, status: str = None) -> dict: ...
```

2.3 Where and How to Host the MCP Server

In enterprise deployments, the MCP Server can be hosted across three distinct topologies:

1. **Local Development / Workstation:** Run as a local process via stdio (`python -m mcp_server.server`) or as an SSE service (`python -m mcp_server.server --sse`). The client launches or connects via localhost loopback with zero network latency.

2. **Docker Containerized Microservice:** Containerized with lightweight Python base (e.g., `python:3.13-slim`), exposing port 8000. Run via `docker run -p 8000:8000 sales-crm-mcp`. Healthcheck endpoint `/healthz` monitors process vitality.
3. **Private Enterprise VPC (AWS ECS Fargate / GCP Cloud Run / K8s):** Deployed in an internal non-public VPC subnet behind an Application Load Balancer with Mutual TLS (mTLS) authentication. Zero egress to public internet ensures complete enterprise data sovereignty.

3. LangGraph Workflow: State, Nodes, and Graph Edges

LangGraph provides a robust, cyclically capable, stateful graph runtime. Unlike naive sequential chains or unconstrained ReAct loops that risk infinite execution, LangGraph enforces deterministic state transitions, rigorous typed schemas, and verifiable guardrail boundaries.

3.1 The Typed State Schema (`SalesAgentState`)

Every node reads from and mutates a centrally governed state schema defined in `agent/state.py`:

State Field	Type	Description & Lifecycle Role
`user_query`	`str`	Raw incoming prompt from salesperson (e.g. 'summary of ABC...')
`target_customer_name`	`Optional[str]`	Customer entity identified by Intent Analyzer ('ABC Corp')
`customer_profile`	`Optional[dict]`	Master metadata from MCP `get_customer()` (contacts, tier, revenue)
`deals`	`List[dict]`	Sales pipeline deals from MCP `get_deals()` (values, stages)
`previous_interactions`	`List[dict]`	Historical meetings & emails from MCP `get_previous_interactions()`
`support_tickets`	`List[dict]`	Tickets from MCP `get_support_tickets()`. Highlights open blockers.
`rag_context`	`Dict[str, Any]`	Multi-domain knowledge: Product specs, Pricing, Case Study X, Playbooks
`recommended_agenda`	`List[str]`	Ranked discussion points prioritizing operational ticket resolution first
`risk_assessment`	`List[str]`	Compliance verification log auditing numerical and agenda integrity
`final_briefing`	`str`	Final markdown briefing synthesized for executive meeting delivery
`audit_trace`	`List[dict]`	Chronological execution log tracking timestamps and node outputs

3.2 Detailed Node Specifications

The workflow consists of five specialized deterministic nodes:

1. **Intent Analyzer Node:** Parses input text to extract customer target and verify user intent (e.g., meeting prep, deal review).
2. **Fetch CRM Data Node (MCP Dispatcher):** Executes JSON-RPC calls against the MCP server for customer profile, deals, interactions, and tickets. Extracts critical flags (e.g. 2 open tickets, Rs 25L deal).
3. **RAG Retrieval Node:** Queries internal vector/lexical knowledge bases for corresponding product specs, pricing tiers, matching industry case studies (Case Study X for BFSI), and sales playbooks.
4. **Sales Strategy Synthesizer Node:** Integrates CRM reality with RAG knowledge to compose the 3-step meeting agenda and executive account overview.
5. **Compliance Guardrail Node:** Verifies that open support tickets are placed FIRST before any product pitch. Checks deal figures and generates audit certifications.

3.3 Graph Edges & Conditional Routing Logic

Routing between nodes is defined in `agent/graph.py` using `workflow.add_edge` and `workflow.add_conditional_edges`. If `fetch_crm` reports an invalid or missing customer account (`mcp_error` present), the graph conditionally short-circuits directly to `END`. Otherwise, execution smoothly proceeds through RAG retrieval, synthesis, and compliance guardrails.

4. RAG Knowledge Integration & The ABC Corp Execution Trace

The RAG Knowledge Engine (`rag_engine/retriever.py`) indexes four vital sales collateral domains: **Product Documentation** (capabilities, latency benchmarks), **Pricing Documentation** (commercial tiers, multi-year terms), **Case Studies** (peer BFSI banking rollouts), and **Sales Playbooks** (CTO alignment strategy).

Real-World Execution Trace: Customer 'ABC Corp'

Here is the exact runtime trace generated when a salesperson requests: *"Give me a summary of customer ABC and tell me what I should discuss in tomorrow's meeting."*

GENERATED EXECUTIVE BRIEFING (FDE Grade)

Customer: ABC Corp (Financial Services & Banking - BFSI)

Primary Stakeholder: Rajesh Sharma (Chief Technology Officer)

Current deal: Rs 25,00,000 (Rs 25L) - Enterprise Core Tier

Last interaction: Sept 28 (Executive Zoom with Priya Nair & Arun Verma)

Open support issues: 2 (TICK-9021: API Batch Timeout; TICK-9034: SSO Token Renewal)

Interested product: Enterprise AI Platform

RECOMMENDED DISCUSSION STRATEGY:

1. **Resolve open support issues first:** Proactively address tickets TICK-9021 and TICK-9034. Present the scheduled Hotfix v2.4.1 raising webhook timeout ceiling to 120s. Proving operational reliability secures executive confidence before contract signing.
2. **Discuss Enterprise AI Core Capabilities:** Revisit latency & data sovereignty requirements raised in the Sept 28 meeting. Reaffirm P99 < 450ms decision latency and private VPC deployment model covered in the Rs 25L tier.
3. **Present Case Study X:** Share the Tier-1 Private Bank deployment where meeting prep time dropped by 92% and the solution passed full RBI regulatory audit without exception.

5. Business Impact & Forward Deployed Engineering (FDE) Value

`92% Reduction in Prep Overhead`	Drops sales prep time from 45 minutes across 4 systems to under 30 seconds of automated briefing.
`Elimination of Blind Spots`	Surfaces critical support ticket friction before high-stakes renewal meetings, preventing contract churn.
`Deal Velocity Acceleration`	Directly equips account executives with tailored peer validation (Case Study X) and accurate pricing tiers (Rs 25L).
`Strict Enterprise Compliance`	Deterministic LangGraph guardrails prevent hallucinations, verify numbers against CRM, and protect brand reputation.